DSCI 551 – Spring 2026
Homework 4: Decoding and Extracting InnoDB Index Records

(100 points)

Due: 11:59pm, March 13, 2026, Friday

NO Late Submissions will be accepted!

1. Overview

In this assignment, you will parse raw InnoDB page data from .ibd files and reconstruct structured index records. Rather than querying through SQL, you will read raw page bytes, decode stored numeric and string values, interpret field metadata from JSON files, and reconstruct full records from storage-level representation.

2. Learning Objectives

- Understand how database records are physically stored.
- Decode signed integers stored using InnoDB encoding.
- Decode floating-point values.
- Extract structured records using metadata.
- Handle multiple SQL data types generically.
- Gain experience with system-level byte parsing.

3. Provided Files

You are given a template notebook (extract_records - template.ipynb), multiple .ibd files, and corresponding decoding .json files. For reference, you are also provided with an SQL script that generates and populates the tables in the test cases.

Your implementation must work for all provided pairs of idb and decoding files.

4. Functions You Must Complete

4.1 decode_innodb_double

This function converts an 8-byte sequence representing a stored double into a Python float.

- Validate that input length is correct.
- Decode using Python's struct module.
- Return a Python float.
- Follow the style used in decode_innodb_float.

4.2 extract_one_record

This function extracts a single record from a page starting at the given offset and returns a dictionary mapping field names to decoded values.

It must properly handle:

- Integer types (TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT) and unsigned versions.
- Floating-point types (FLOAT, DOUBLE).
- Character types (CHAR, VARCHAR, TEXT).
- NULL values.
- Other data types by returning raw bytes as a hex string (e.g., 0x...).

The function must extract fields in order, use metadata for size and type, call appropriate decoding helpers, and advance offsets correctly.

5. Understanding Signed Integer Decoding

Important: The function decode_innodb_signed_int is fully implemented in the template. You do NOT need to modify this function. However, you are expected to understand how it works and use it correctly when decoding integer fields in extract_one_record.

Note that InnoDB stores signed integers in a modified format. First, values are represented in normal two's complement format, then the most significant bit is flipped before storage. As a result, the stored value needs to be first read as unsigned, flip its sign bit, and then converted into the proper signed value (see codes for more details).

6. Understanding Signed Integer Decoding Logic

Based on the provided function: decode_innodb_signed_int, answer the following questions (30 points total):

1. Why are only specific byte lengths supported?
2. Why must the number of bits be computed from the byte length?
3. Why is the value first read as an unsigned integer?
4. What does the sign mask (1 << (bits - 1)) represent?
5. What effect does the XOR operation have on the stored value?
6. Why might InnoDB flip the sign bit before storing signed integers?
7. Why do we compare against (1 << (bits - 1)) before subtracting?
8. How does subtracting (1 << bits) convert the value into the correct signed range?
9. What would happen if the sign-bit flip step were skipped?
10. Explain the full transformation from stored bytes to final signed integer.

7. Testing Requirements

Your solution must work for all provided file pairs, for example:

```
result = extract_indexes_from_ibd('user.ibd', 'user_decoding.json')
print(json.dumps(result, indent=2))
```

Your output should match exactly with the one provided in the template. Your implementation must not crash and must correctly decode supported types.

8. Grading Rubric

- decode_innodb_double implementation — 10 points
- extract_one_record implementation — 60 points
- Conceptual questions: Understanding signed integer decoding — 30 points
- Total — 100 points

9. Submission Instructions

In addition to completing the code, you must include written answers to the conceptual questions at the end of your submitted .ipynb file. Place your answers in clearly labeled Markdown cells after your code.

You must submit your completed Jupyter notebook (.ipynb file).

Before submitting:

- Remove the word 'template' from the original filename.
- Prepend your full name to the filename.

Example: JaneDoe_extract_records.ipynb

Appendix A — Understanding the Decoding JSON Files

Each decoding JSON file describes how index records are physically stored inside a corresponding .ibd file. Your extraction logic must rely on this metadata.

- index_id — Internal InnoDB identifier for the index.
- name — Name of the index (e.g., PRIMARY).
- records — Field layouts describing record structure.

Each field definition includes:

- name — Column name.
- type — Logical SQL data type.
- size — Number of bytes stored in the record.
- is_null — Whether the field can be NULL.

Important observations:

- System fields such as DB_TRX_ID and DB_ROLL_PTR must also be parsed.
- Field sizes may vary across records (e.g., VARCHAR).
- Field order in JSON matches physical storage order.
- Extraction logic must rely on metadata, not hardcoded assumptions.

Appendix B — Useful Resources

- InnoDB Row Formats: https://dev.mysql.com/doc/refman/8.4/en/innodb-row-format.html
- Data Types: https://dev.mysql.com/doc/refman/8.4/en/data-types.html

- The basics of InnoDB space file layout: https://blog.jcole.us/2013/01/03/the-basics-of-innodb-space-file-layout/
- InnoDB index record format: https://blog.jcole.us/2013/01/10/the-physical-structure-of-records-in-innodb/
- The physical structure of InnoDB index pages: https://blog.jcole.us/2013/01/07/the-physical-structure-of-innodb-index-pages/
- B+Tree index structures in InnoDB: https://blog.jcole.us/2013/01/10/btree-index-structures-in-innodb/
- Page management in InnoDB space files: https://blog.jcole.us/2013/01/04/page-management-in-innodb-space-files/
- Efficiently traversing InnoDB B+Trees with the page directory: https://blog.jcole.us/2013/01/14/efficiently-traversing-innodb-btrees-with-the-page-directory/
- Two's complement: https://www.cs.cornell.edu/~tomf/notes/cps104/twoscomp.html#:~:text=Suppose%20we're%20working%20with,Conversion%20to%20Two's%20Complement
- Big vs little endian: https://en.wikipedia.org/wiki/Endianness